

Manhwa / Webtoon Recap Pipeline

Deep research on panel slicing, stylized-text OCR, ordering, preprocessing and downstream components - with manga-only models excluded from the main recommendations

Research date: 24 September 2026 **Target:** long vertical webtoon strips with diagonal, overlapping, borderless and full-bleed panels; stylized English text including dialogue, narration and sound effects.

Executive finding: I could not find a publicly validated, tiny, off-the-shelf panel model that is proven on your exact hardest layout class. The strongest route is a hybrid: cheap gutter/geometry pass for easy regions, learned instance segmentation for hard panels, mask refinement when necessary, and a separate arbitrary-shape text detector + compact scene-text recognizer. That avoids forcing one model to solve four different problems.

1. What counts as a useful candidate

I filtered for public or at least verifiable implementations, real model cards or papers, current repository activity where possible, and a concrete reason the method matches your layout/text failure modes. I treated **bbox-only panel detectors**, **manga-trained OCR**, and generic document OCR that has no evidence of art/scene-text robustness as secondary at best. The report separates deployable tools from training references and benchmark-only products.

2. Panel slicer / segmentation

Candidate	What it gives	Fit to your hard case	Status
Segment Anything Comic	Polygon masks for comic frames; runnable local app; Apache-2.0.	Very relevant because it explicitly predicts frame polygons rather than forcing rectangles. Best research base for diagonal/irregular panels.	Use / fine-tune
TritonPanels	Western-comic-trained on-device detector in a real reader app.	Current changelog specifically claims gains on full-bleed, borderless digital layouts and busy grids. No weights, so it is a benchmark/reference, not a drop-in model.	Benchmark only
RF-DETR Seg Small / Medium	General instance segmentation architecture; Small to Medium are 4.4M / 5.9M params in the published model table.	Not a panel model out of the box. Excellent candidate for your own polygon-mask fine-tune while staying small.	Train on your data
EdgeSAM	Promptable segmentation optimized for edge; ONNX export.	Useful as a mask refiner after a panel proposal exists. It does not understand "panel" semantics by itself.	Refiner
MobileSAM	9.66M total params; official repo reports about 12ms/image on a single GPU for prompted segmentation.	Good cheap fallback/refiner. Still class-agnostic, so it needs prompts or proposal generation.	Refiner / fallback
FastSAM	Class-agnostic segmentation trained on 2% of SA-1B; designed for speed.	Useful for fast mask proposals but not a panel understanding model. Expect post-processing and filtering work.	Proposal stage
Kumiko / PST	Classical OpenCV gutter/contour/Hough approaches.	Very cheap for easy pages. Both projects document limitations around borderless/irregular cases, so use them as a front-end rather than the final judge.	Fast path
Comic Panel Extractor	2026 classical pipeline with thresholding, morphology, contours, NMS and ordering.	Useful baseline but its own documentation says overlapping, non-rectangular/splash and zero-gutter pages can fail.	Baseline

Best architecture for your exact layout: do not crop by bbox alone. Store polygon_mask + bbox + confidence + page_id + reading_order. Generate a rectangular crop only at the very end by masking or tight-padding the polygon. This prevents a diagonal panel from dragging half the neighboring panel into the crop.

3. The most interesting non-manga panel research

Segment Anything Comic is the most important find here. Its repository is explicitly modified to predict polygon segmentations of comic frames, provides an app, checkpoints and training code, and is Apache-2.0. It was not presented as a webtoon-specific commercial product, but its output type matches your failure mode unusually well. ■cite not rendered■

TritonPanels is valuable for a different reason: it is evidence from a current product that a *Western-comic-trained* detector is being tuned for full-bleed, borderless and busy layouts. The public changelog is not enough to establish model accuracy or reproduce it, so treat it as a benchmark target rather than proof of a reusable checkpoint.

RF-DETR Seg is the strongest small general-purpose training base I found. The official model table exposes a 4.4M-parameter small segmentation model and 5.9M medium model. Because the architecture is generic, you can train it on your own polygon panel masks rather than inheriting a manga-biased prior.

4. OCR: the part that will decide whether your recap is readable

For stylized manhwa text, the right mental model is **text spotting**, not document OCR. A pipeline should first find arbitrary-shaped text instances, then recognize the cropped text with a recognizer that has visual + language context. Sound effects and stylized lettering are often the cases where a clean-looking detector plus weak recognizer turns into transcript garbage.

Candidate	Role	Evidence	Pipeline use
SVTRv2-T / S	Fast English scene-text recognition	Official OpenOCR table lists T at 5.13 size units and S at 11.25; common-benchmark average 94.78 for T and 96.13 for S. The paper explicitly tests artistic, multi-oriented and long text.	Hot-path recognizer; S if quality matters more than size
PARSeq Tiny	Tiny Latin scene-text recognizer	128x32 input; 62 case-sensitive alphanumerics + 32 punctuation. The family is designed to combine context-free and context-aware decoding.	Fast recognizer for short bubbles/SFX
PARSeq full	Higher-quality scene-text recognition	Reference repo reports 95.95 average on combined common benchmarks for the full model, 23.8M params and 14.87ms median in its benchmark setup; supports ONNX export.	Quality lane / disagreement resolver
COMICS Text+	Comic-specific text detection + recognition training stack	Fine-tuned FCENet detector and MASTER recognizer trained on custom comics data; the project exposes data/configs and an end-to-end text extractor.	Best training reference for comic text
DeepSolo	Arbitrary-shape text spotting	Single decoder predicts text geometry and recognition; works with line annotations and is designed for arbitrary-shaped text.	Text detector/spotter for diagonal or irregular lettering
TextSnake / DPText-DETR	Arbitrary-shaped text detection	TextSnake explicitly models horizontal, oriented and curved text. DPText-DETR uses point-based dynamic queries for scene text.	Detection before OCR on stylized text
MaskTextSpotterV3	Segmentation-based text spotting	Official repo says it improves robustness to rotations, aspect ratios and shapes.	Fallback detector when text geometry is ugly
RapidOCR + PP-OCR models	Fast ONNX runtime / engineering layer	RapidOCR is a current multi-platform ONNX-based deployment toolkit. PP-OCRv6 has tiny/small/medium tiers in the current PaddleOCR release.	Production runtime / cheap baseline
PaddleOCR-VL-1.6	Heavy quality fallback / OCR oracle	Current release reports 96.33 on OmniDocBench v1.6 and adds stronger text spotting. 0.9B class model.	Only run on low-confidence crops

5. Why your previous OCR outputs probably turned into garbage

The most common architecture mistake is sending the full panel to OCR and asking a generic recognizer to discover all text, speaker groups, orientation and reading order simultaneously. For stylized manhwa, that creates false positives from line art, background objects and SFX. The stronger design is **detect text instances first**, then crop each instance tightly, normalize its geometry, recognize it, and use a slower recognizer only when the fast recognizers disagree or report low confidence.

Recommended confidence cascade: DeepSolo/FCENet/DB-style detector -> crop polygon/quad -> SVTRv2-T or PARSeq-Tiny -> if low confidence or disagreement -> PARSeq full -> if still bad -> PaddleOCR-VL-1.6 or a local VLM. Keep the original image and the raw OCR for audibility. Do not silently “correct” names or invented words without retaining the raw reading.

6. Other pipeline components that are actually useful

A good recap pipeline is not just slicer + OCR. The painful errors move downstream unless you model them explicitly.

Component	Recommended pattern
Long-strip tiling / reslicing	Split huge vertical strips into overlapping working windows before running vision models. Preserve a shared page coordinate system. Overlap windows enough that bubbles near a seam appear completely in at least one window. Reproject all detections to global coordinates and merge by IoU/polygon overlap.
Panel instance separation	Use instance masks, not semantic "panel pixels". Two touching panels need two instances even when their artwork is visually continuous.
Polygon-to-crop renderer	Mask the panel polygon against the original image, compute a tight alpha crop, then pad a small amount. This is what stops diagonal panels from importing neighboring content.
Reading order	Use deterministic geometry first. Only invoke a vision model on pages whose graph has ambiguous crossings, overlaps or inset panels. Store both the geometric order and the resolver decision.
Text grouping	Cluster detected text lines into bubble-level groups by overlap, distance and shared orientation. Recognition should happen per line or compact group, not on a giant panel.
SFX / narration / dialogue typing	Keep separate classes in your metadata even if the recap ultimately uses only dialogue. A sound effect detector can otherwise pollute the transcript.
Scene grouping	ComicScene154 is a useful Western-comic benchmark showing that scene segmentation is a distinct problem above panel order. Group panels into narrative scenes before generating recap prose so one prompt does not contain unrelated events.
OCR consensus	For hard text, compare two recognizers. If both agree, accept. If they disagree, send only that crop to the heavy fallback. This keeps cost and latency bounded.
Human review queue	Anything with low panel confidence, tiny text, severe overlap, or recognizer disagreement should become a review item. A 98% automatic system with a 2% catastrophic-error rate can still be painful when that 2% is the protagonist's name in every chapter.

7. A concrete pipeline I would prototype

Stage A - ingest: decode the chapter, remove duplicate frames/logos/ads only if your source includes them, preserve original page coordinates.

Stage B - cheap structure pass: run gutter/projection/contour logic. Accept only high-confidence rectangular or clearly separated areas.

Stage C - hard panel pass: for low-confidence regions, use a custom RF-DETR Seg Small/Medium model trained on your hand-labeled polygon masks. For especially difficult masks, refine with EdgeSAM or MobileSAM.

Stage D - panel crop: crop from the polygon mask, not the bbox. Preserve a small padding margin and attach source coordinates.

Stage E - text spotting: DeepSolo / FCENet / another arbitrary-shape detector -> text polygons -> per-text crop.

Stage F - fast OCR: SVTRv2-S or PARSeq-Tiny on every crop. Keep confidence / alternative outputs.

Stage G - quality OCR: PARSeq full for disagreements; PaddleOCR-VL-1.6 only for the small set that still fails.

Stage H - structure: classify text as dialogue, narration, SFX, sign, UI/logo; group lines into bubbles; restore reading order.

Stage I - recap prep: group panels into scenes, attach OCR text plus panel images, then generate the recap narrative from structured scene packets rather than one giant raw transcript.

Stage J - review: review only the uncertain panels/text boxes. Export both machine output and corrected output for future fine-tuning.

Key engineering idea: your manually sliced examples are more valuable than another random pretrained checkpoint. Use them twice: first to benchmark existing detectors, then to fine-tune a very small segmentation model. The report's findings strongly suggest that domain-specific geometry is the bottleneck, not raw parameter count.

8. Candidates I would actually benchmark first

#	Stage	Candidate	Why
1	Panel geometry	Custom RF-DETR Seg Small/Medium	Train on your own polygon masks. This is the best route to a tiny production model.
2	Panel geometry	Segment Anything Comic	Use as the irregular-panel research baseline and teacher/refinement reference.
3	Panel geometry	EdgeSAM / MobileSAM	Use only as mask refinement / prompt-based fallback, not the panel detector itself.
4	Easy page splitter	Kumiko / classical gutter pass	Use for free speed on easy areas and to reduce expensive inference.
5	OCR detector	DeepSolo or FCENet	Choose arbitrary-shape text spotting over generic document boxes.
6	OCR recognizer	SVTRv2-S	Good speed/quality balance with explicit artistic and multi-oriented benchmarks.
7	OCR recognizer	PARSeq Tiny	Use as a second fast recognizer for consensus or as the cheapest lane.
8	OCR recognizer	PARSeq full	Use only on uncertain crops.
9	OCR fallback	PaddleOCR-VL-1.6	Heavy but useful as a low-volume rescue model.
10	Runtime	RapidOCR / ONNX	Use as an engineering layer where supported models can be exported or swapped.
11	Scene grouping	ComicScene154 methodology	Useful benchmark for the scene stage, especially if you want recap units larger than individual panels.

9. Things I deliberately did NOT recommend as your main answer

Manga-only OCR or segmentation checkpoints were excluded from the main shortlist because your target is manhwa/webtoon English recap and you specifically asked for non-manga sources.

Generic document OCR as the only OCR stage was also not treated as sufficient. Document OCR can be excellent on clean pages while still missing stylized text over artwork, rotated labels, or irregular SFX.

Pure bbox panel detectors were treated as insufficient for the hardest cases. A diagonal or overlapping panel can require a polygon or mask to prevent neighboring content from leaking into the crop.

10. Reality check on “accuracy”

Published numbers are not directly comparable across repositories. Some are on comic-specific validation sets, some on natural-scene text datasets, some on document benchmarks, and some are self-reported product claims. Treat them as evidence about model behavior, not a universal leaderboard. For your project, the only metric that really matters is a benchmark built from your own manually sliced and transcribed webtoon samples.

Suggested benchmark: 100 hard panels, stratified into diagonal, overlapping, borderless, full-bleed, tiny text, stylized font, SFX, narration, and text-over-art categories. Track panel IoU / boundary error separately from OCR character error rate and exact-match. Report the 50th and 95th percentile latency, not just average FPS.

11. Source notes and links

The links below are the primary public sources consulted for this report. Most are GitHub repositories or official model cards. The report does not claim that every repository is production-ready; it records what the project itself actually exposes and where its documented limitations matter.

Vrroom - Segment Anything Comic

<https://github.com/Vrroom/segment-anything-comic>

Polygon segmentation of comic frames; runnable app; Apache-2.0.

TritonPanels changelog

<https://tritonpanels.tritonapps.com/changelog/>

Current Western-comic-trained panel detector claims full-bleed, borderless digital layouts and busy grids; proprietary/no weights.

TritonPanels panel-by-panel guide

<https://tritonpanels.tritonapps.com/guides/panel-by-panel-comic-reading>

Explains on-device detector and Western training focus.

Roboflow RF-DETR

<https://github.com/roboflow/rf-detr>

General instance segmentation with Small/Medium/Large/XLarge/2XL variants; good base for custom panel masks.

MobileSAM

<https://github.com/ChaoningZhang/MobileSAM>

9.66M total parameters; reported 12ms/image on a single GPU for prompted segmentation; ONNX export.

EdgeSAM

<https://github.com/chongzhou96/EdgeSAM>

SAM variant for edge deployment; ONNX; reports large speedups over original SAM.

FastSAM

<https://github.com/CASIA-IVA-Lab/FastSAM>

Lightweight class-agnostic segmentation trained on 2% of SA-1B; useful as a fast mask proposal stage, not panel semantics.

Kumiko

<https://github.com/njean42/kumiko>

Classical CV comic cutter; arbitrary-shape detection but exports rectangles; roadmap explicitly notes non-framed/intertwined limitations.

Comic Panel Segmentation Tool

<https://github.com/reidenong/ComicPanelSegmentation>

Classical CV using Canny/Hough; very lightweight but depends on visible delineations.

Comic Panel Extractor

<https://github.com/ehsanx/Comic-Panel-Extractor>

2026 classical pipeline; Otsu/adaptive threshold, morphology, contours, NMS, reading-order sorting; documented failure on overlap, non-rectangular and zero-gutter panels.

Panelizer

<https://github.com/hummat/panelizer>

Useful architecture: cheap CV first, ML fallback, VLM only for ambiguous reading order, human overrides.

gsoykan comics_text_plus

https://github.com/gsoykan/comics_text_plus

Western-comic OCR research; FCENet detector and MASTER recognizer fine-tuned on custom comics data; public train/test assets.

DeepSolo

<https://github.com/ViTAE-Transformer/DeepSolo>

General scene-text spotter; explicit points; handles arbitrary-shaped text and recognition together.

TextSnake

<https://github.com/princewang1994/TextSnake.pytorch>

Arbitrary-shape text detection; explicitly supports horizontal, oriented and curved text.

DPTText-DETR

<https://github.com/ymy-k/DPTText-DETR>

Point-based transformer for arbitrary-shape scene text detection; useful for stylized/rotated text regions.

MaskTextSpotterV3

<https://github.com/MhLiao/MaskTextSpotterV3>

Segmentation proposal text spotting; explicitly improves robustness to rotations, aspect ratios and shapes.

SVTRv2 / OpenOCR

<https://github.com/Topdu/OpenOCR>

Very small STR models; SVTRv2-T 5.13 units reported model size and fast latency; strong benchmark results with artistic/multi-oriented metrics exposed.

PARSeq

<https://github.com/baudm/parseq>

Scene text recognizer with internal language modeling; 95.95 average on combined common benchmarks for the full model; ONNX export possible; tiny variant available.

RapidOCR

<https://github.com/RapidAI/RapidOCR>

ONNX Runtime based fast OCR deployment layer; supports English and multiple platforms.

PaddleOCR official

<https://github.com/PaddlePaddle/PaddleOCR>

Current PP-OCRv6 family has tiny/small/medium tiers and strong general scene-text support; also includes PaddleOCR-VL.

PaddleOCR-VL-1.6

<https://github.com/PaddlePaddle/PaddleOCR>

0.9B compact document VLM; current release reported 96.33 on OmniDocBench v1.6; better as fallback/oracle than hot-path OCR.

Surya

<https://github.com/datalab-to/surya>

650M OCR/layout model with reading order; useful fallback/validator but heavier than the requested lightweight hot path.

COMICS Text+ / IMCDB

<https://github.com/gesstalt/IMCDB>

English comic dataset with 2,303 pages, 9,212 panels, 18,424 text files and balloon/narration masks, per ICDAR 2021.

ComicScene154

<https://github.com/KnorrscHe/ComicScene154>

Western public-domain comics scene segmentation dataset with scene/panel annotations and benchmark code.

ChROMIC

<https://github.com/daydayup586/ChROMIC>

Benchmark for panel reordering and narrative reasoning using Western and Japanese comics; useful for evaluating reading-order logic.

Mask TextSpotter

<https://github.com/MhLiao/MaskTextSpotter>

Arbitrary-shape end-to-end text spotting with masks; older but conceptually strong for text over artwork.

ABCNet v2 paper

<https://arxiv.org/abs/2105.03620>

Bezier-curve text spotting; designed for arbitrary-shaped text with high efficiency.

SVTRv2 paper

<https://arxiv.org/abs/2411.15858>

ICCV 2025; focuses on irregular text and speed, with ablations across artistic, multi-oriented and long-text settings.

Panel extraction review

<https://www.sciencedirect.com/science/article/pii/S0952197623018997>

Systematic literature review covering 72 comic-recognition papers through 2023.

12. Bottom line

For your specific pipeline, the research points away from a single “magic slicer” and toward a **confidence-gated hybrid**. The hidden-gem combination is: classical gutter splitting for cheap wins -> a tiny custom polygon segmentation model for the actual hard pages -> lightweight arbitrary-shape text spotting -> SVTRv2/PARSeq fast OCR -> a heavy fallback only on disagreement -> scene grouping before recap generation. That setup is more work than installing one checkpoint, because apparently computers have decided that panel geometry deserves an entire career, but it is the path most consistent with the evidence I found.